

Rollno :241001307

IMPLEMENTING A* ALGORITHM

Python Code:

```
from queue import PriorityQueue

def a_star(graph, start, goal, heuristic):

    open_set = PriorityQueue()
    open_set.put((0, start))

    came_from = {}

    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    while not open_set.empty():

        _, current = open_set.get()

        if current == goal:
```

```
path = []

while current in came_from:
    path.append(current)
    current = came_from[current]

path.append(start)

return path[::-1]

for neighbor, cost in graph[current]:

    tentative_g = g_score[current] + cost

    if tentative_g < g_score[neighbor]:

        came_from[neighbor] = current
        g_score[neighbor] = tentative_g

        f = tentative_g + heuristic[neighbor]

        open_set.put((f, neighbor))

return None
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('D', 2), ('E', 5)],  
    'C': [('F', 1)],  
    'D': [('G', 2)],  
    'E': [('G', 1)],  
    'F': [('G', 3)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 2,  
    'D': 1,  
    'E': 0,  
    'F': 3,  
    'G': 0  
}
```

```
path = a_star(graph, 'A', 'G', heuristic)
```

```
print("Shortest path using A*:", path)
```

Output:

Shortest path using A*: ['A', 'B', 'D', 'G']

Result:

Thus the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree was executed successfully.